

Aqua-Net: A Deep Dive into Aquatic Life

MIS 6900: Deep Learning and Text Analytics

Presented To:

Prof. Vijayan Sugumaran

Presented By:

Jigyasa Soni, Khyati Sharma & Vishaka Sharma

Table of Contents

Introduction	2
Problem Statement	3
Business Case/Need	3
Data Collection	3
Exploratory Data Analysis	4
Model Building.....	5
Data Augmentation	9
Transfer Learning	12
Comparison.....	14
Model Application.....	15
Conclusion	15
Challenges.....	16
References	17

Introduction

This deep learning project is to leverage the advancements in technology and data processing to tackle real-world challenges, particularly in the field of image classification. Image classification has become a core task in computer vision, with applications spanning across various industries, from healthcare to wildlife conservation. In this project, we focus on classifying aquatic wildlife images using Convolutional Neural Networks (CNNs). By employing CNNs, we aim to effectively identify and classify different species of sea animals, highlighting the strengths of these networks in image recognition.

In our approach, we incorporated data augmentation techniques, such as rotation, flipping, and scaling, to enhance the model's generalization ability and improve its accuracy. These techniques simulate variations in image quality, illumination, and size, mimicking real-world scenarios. Additionally, by utilizing transfer learning, we aim to optimize the deep learning model, achieving high classification performance on real-world aquatic datasets. This project, **Aqua-Net: A Deep Dive into Aquatic Life**, demonstrate how deep learning, specifically CNNs, can be applied to solve complex classification problems and lay the groundwork for future research and developments in the field of computer vision.

Problem Statement

The identification and classification of aquatic life, particularly sea animals, represent a significant challenge within the field of computer vision and deep learning. The variability in image quality, lighting conditions, and the wide range of species poses difficulties in developing an accurate and generalized model.

In this project, we aim to develop a robust Convolutional Neural Network (CNN) for the classification of sea animals, specifically focusing on three species: **dolphins, jellyfish, and sea turtles**. The core objective is to design a deep learning model that can accurately classify these species under varying environmental conditions, leveraging CNN's capacity to extract hierarchical features from images.

This project seeks to demonstrate the potential of CNNs in addressing the challenges of marine species classification, showcasing the effectiveness of data augmentation, transfer learning, and deep learning techniques in building a reliable and scalable solution for real-world applications in marine biology and biodiversity conservation.

Business Case/Need

Sea animals play a crucial role in marine ecosystems and understanding their populations is essential for conservation efforts. Automated classification of sea animal images helps in laying focus on species identification, population tracking, and habitat monitoring. Targeted use of a

focused model with a computer-aided categorization with focused monitoring would act as an asset in understanding while also establishing alerts on monitoring the effects of diverse conservation strategies.

Data Collection

The dataset for this project was obtained from [Kaggle](#), initially containing twenty-three categories. As part of the preprocessing process, we reduced the dataset to focus on three specific species categories. This reduction was done to streamline the classification task and provide a clearer foundation for model development, allowing us to better handle the complexities of the data. These three categories were selected based on their relevance to marine biology research and their distinct visual features, making them ideal for building a model that is easier to scale and adapt for future, larger datasets.

Exploratory Data Analysis

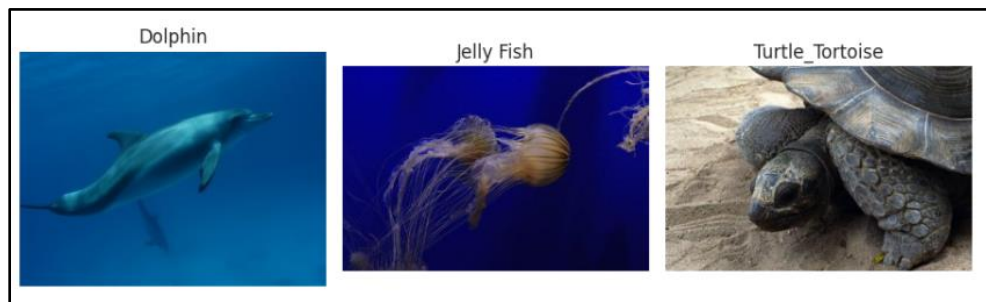
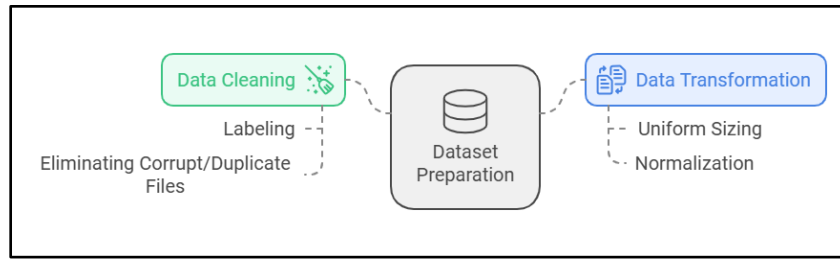
The dataset used in this project consists of images from 23 different species of sea animals. However, the focus was narrowed to three selected species: Jellyfish, Dolphin, and Sea Turtle. To train and evaluate the model effectively, we split the dataset into three distinct subsets: the training set, validation set, and test set.

```
TRAIN SET:
  Turtle_Tortoise: 1332 images
  Dolphin: 547 images
  Jelly Fish: 591 images

TEST SET:
  Turtle_Tortoise: 286 images
  Dolphin: 118 images
  Jelly Fish: 127 images

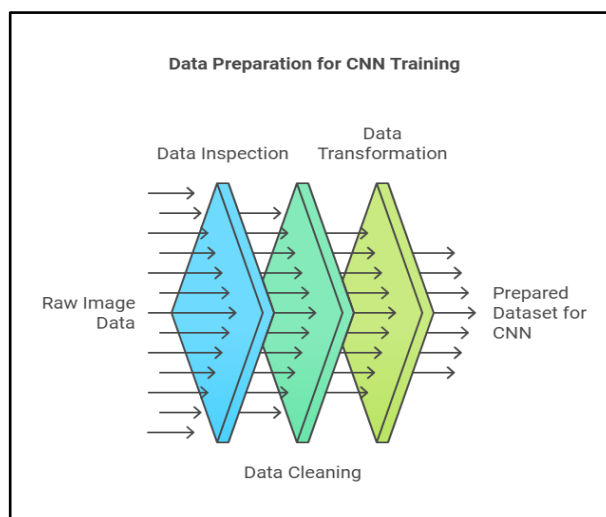
VAL SET:
  Turtle_Tortoise: 285 images
  Dolphin: 117 images
  Jelly Fish: 127 images
```

The dataset posed several challenges, including variations in image resolutions, aspect ratios, lighting conditions, and complex backgrounds. These factors could potentially hinder the model's performance, so several preprocessing steps were implemented to standardize the images and enhance the model's generalization ability.



To address the variability in pixel intensities, we performed a statistical analysis of the pixel intensity distributions and normalized the data, ensuring that the most relevant features were preserved while minimizing unnecessary variations. Feature extraction was optimized to highlight animal-specific attributes, such as textures, shapes, and patterns, which are critical for classification.

We evaluated classification performance using a confusion matrix, where class-wise accuracy metrics were reviewed to identify which species were most frequently misclassified. This analysis provided insight into areas of improvement and helped refine the model. Through these preprocessing and optimization techniques, the dataset was adequately prepared, ensuring that it was suitable for training a Convolutional Neural Network (CNN). Ultimately, these efforts aimed to improve the model's ability to perform well on real-world scenarios, achieving high classification accuracy.



Model Building

The model built is a **Convolutional Neural Network (CNN)** designed for a multi-class image classification task with three possible classes. It processes input images, extracts relevant features, and classifies them into one of these three classes. The model architecture uses a series of convolutional layers (Conv2D) with progressively increasing filter sizes (**32, 64, and 128**) to extract higher-level features from input images. Each convolutional layer is followed by **BatchNormalization**, which normalizes activations to stabilize training, and **MaxPooling2D layers**, which reduce the spatial dimensions of the feature maps. After the convolutional blocks, the output is flattened into a one-dimensional vector, passed through a dense layer with 128 units, and then processed through a final dense layer with 3 units and a **softmax activation function for multi-class classification**.

The model is compiled with the **Adam optimizer**, **categorical cross-entropy loss**, and **accuracy as the evaluation metric** to handle a multi-class classification problem.

Model: "sequential"		
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 32)	896
batch_normalization (BatchNormalization)	(None, 224, 224, 32)	128
batch_normalization_1 (BatchNormalization)	(None, 224, 224, 32)	128
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_1 (Conv2D)	(None, 112, 112, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 112, 112, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_2 (Conv2D)	(None, 56, 56, 128)	73,856
batch_normalization_3 (BatchNormalization)	(None, 56, 56, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 128)	0
flatten (Flatten)	(None, 100352)	0
dense (Dense)	(None, 128)	12,845,184
batch_normalization_4 (BatchNormalization)	(None, 128)	512

dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 3)	387
Total params: 12,940,355 (49.36 MB)		
Trainable params: 12,939,587 (49.36 MB)		
Non-trainable params: 768 (3.00 KB)		

The model is trained for up to **40 epochs** using `train_gen` and `valid_gen` as data generators for the training and validation sets. Shuffling is turned off, meaning data will be processed in a fixed

order. **MirroredStrategy** is employed for distributed training across multiple GPUs, enhancing the efficiency and speed of the training process.

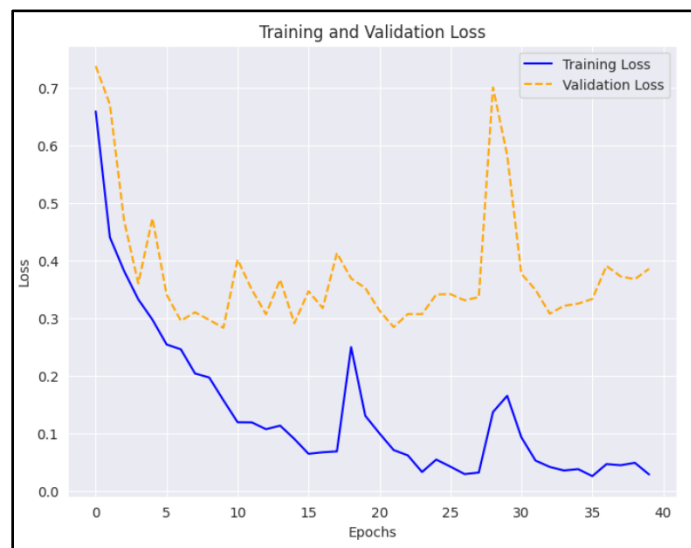
[illegible]

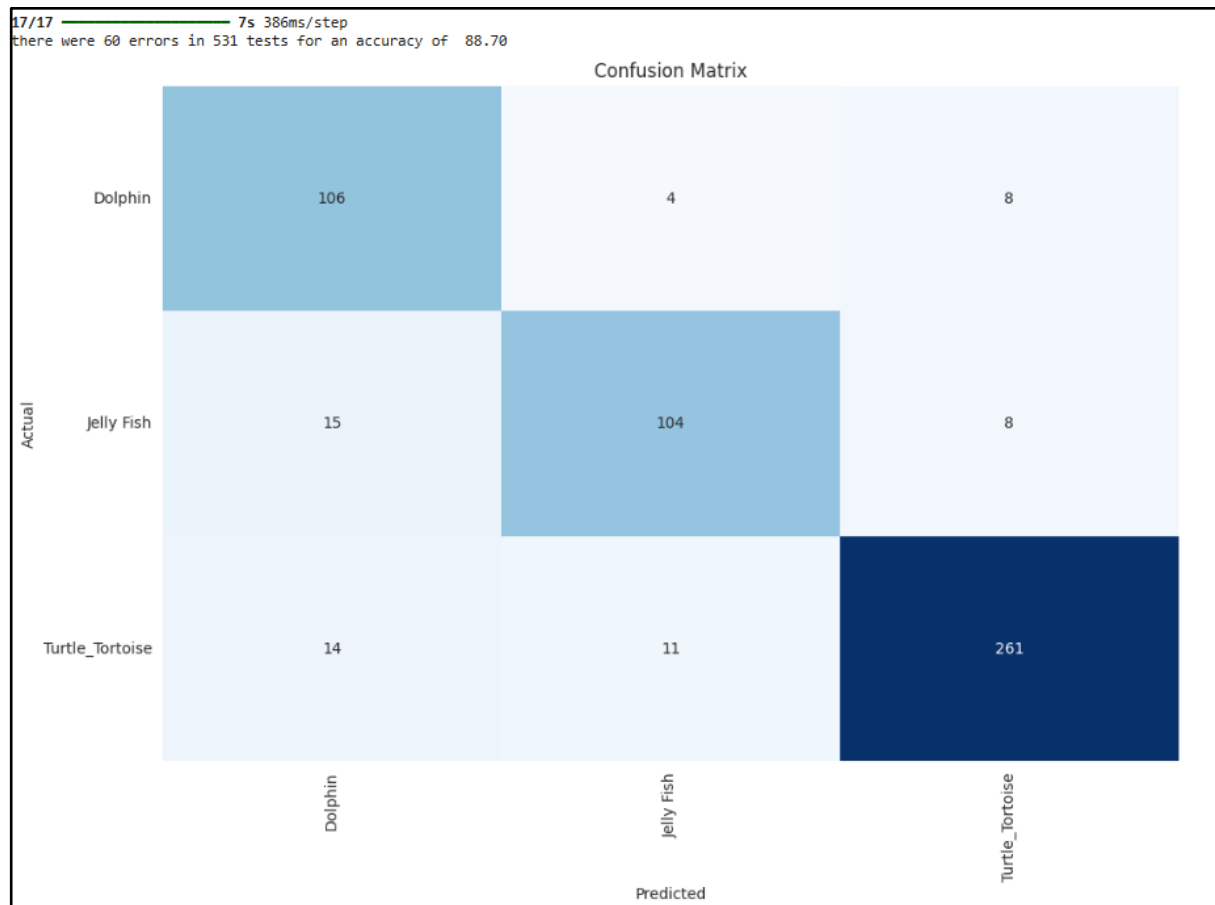
```
Epoch 23/40
78/78 ————— 20s 254ms/step - accuracy: 0.9799 - loss: 0.0570 - val_accuracy: 0.9187 - val_loss: 0.3073
Epoch 24/40
78/78 ————— 18s 224ms/step - accuracy: 0.9945 - loss: 0.0325 - val_accuracy: 0.9206 - val_loss: 0.3072
Epoch 25/40
```

Epoch 24 marks the point where the model achieved the optimal balance between training performance and generalization, with **strong accuracy and minimal loss** on both the training and validation sets.

The model reached **99.45% accuracy on the training data**, indicating excellent learning and near-perfect classification of the training samples. The **training loss** is **0.0325**, reflecting a very low error on the training set and suggesting that the model had effectively minimized the error during training.

At **92.06% validation accuracy**, the model performed well on the **validation set**, indicating that it generalizes effectively to unseen data. While there is a slight gap between the training and validation accuracy. The **validation loss** of **0.3072** is quite low, indicating that the model's predictions on the validation data are close to the true values, with minimal error.





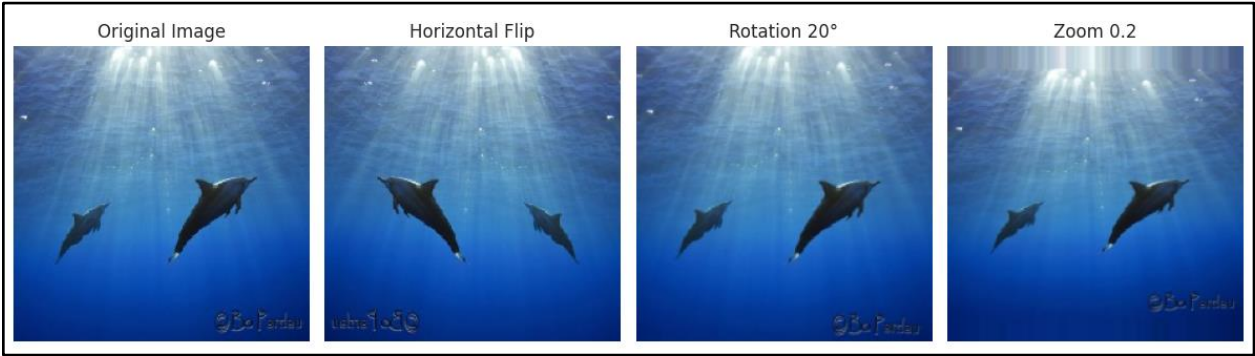
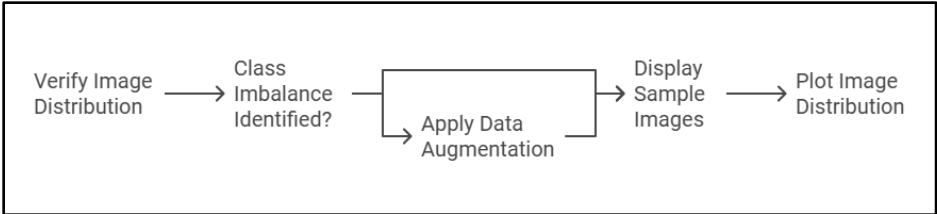
Classification Report:

	precision	recall	f1-score	support
Dolphin	0.7852	0.8983	0.8379	118
Jelly Fish	0.8739	0.8189	0.8455	127
Turtle_Tortoise	0.9422	0.9126	0.9272	286
accuracy			0.8870	531

The model performs well and is highly accurate on both the validation and test sets. **It excels in classifying "Turtle/Tortoise" but could be further improved in identifying "Dolphin" and "Jelly Fish."**

Data Augmentation

The data augmentation techniques are utilized to address class imbalance and improve generalization. The data is processed using ImageDataGenerator from TensorFlow Keras. For the training set, augmentations are applied during the image loading process, while the validation and test sets are only rescaled to normalize pixel values. The images are resized to a target size of (224, 224), and batch processing is performed to handle large datasets efficiently. The class labels are encoded in a one-hot format for multi-class classification. **The specific techniques used are, horizontal flipping, a rotation range of 20 degrees, and a zoom range of 0.2.**



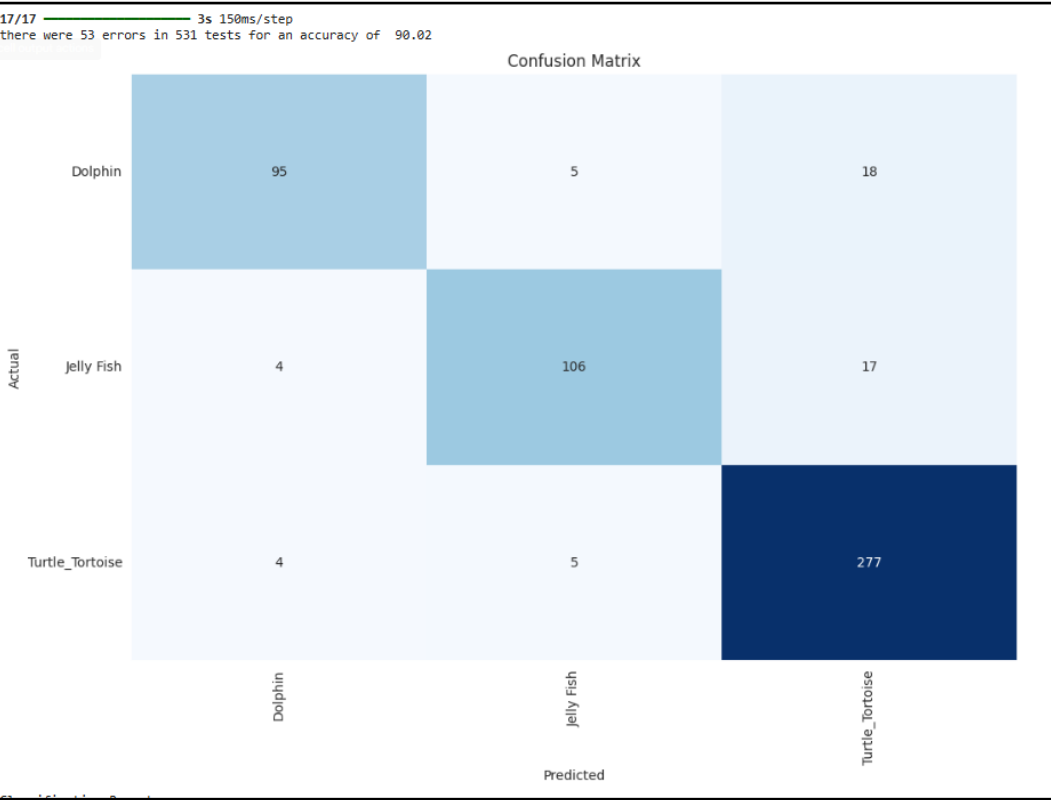
Model: "sequential_2"		
Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 224, 224, 32)	896
batch_normalization_10 (BatchNormalization)	(None, 224, 224, 32)	128
batch_normalization_11 (BatchNormalization)	(None, 224, 224, 32)	128
max_pooling2d_6 (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_7 (Conv2D)	(None, 112, 112, 64)	18,496
batch_normalization_12 (BatchNormalization)	(None, 112, 112, 64)	256
max_pooling2d_7 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_8 (Conv2D)	(None, 56, 56, 128)	73,856
batch_normalization_13 (BatchNormalization)	(None, 56, 56, 128)	512
max_pooling2d_8 (MaxPooling2D)	(None, 28, 28, 128)	0
flatten_2 (Flatten)	(None, 100352)	0
dense_4 (Dense)	(None, 128)	12,845,184
batch_normalization_14 (BatchNormalization)	(None, 128)	512
dropout_2 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 3)	387

Total params: 12,940,355 (49.36 MB)
Trainable params: 12,939,587 (49.36 MB)
Non-trainable params: 768 (3.00 KB)

```
Epoch 34/40
78/78 ————— 54s 658ms/step - accuracy: 0.9120 - loss: 0.2407 - val_accuracy: 0.9357 - val_loss: 0.2122
Epoch 35/40
78/78 ————— 45s 541ms/step - accuracy: 0.9057 - loss: 0.2694 - val_accuracy: 0.9055 - val_loss: 0.2432
Epoch 36/40
78/78 ————— 46s 546ms/step - accuracy: 0.9137 - loss: 0.2401 - val_accuracy: 0.9187 - val_loss: 0.2441
Epoch 37/40
78/78 ————— 50s 604ms/step - accuracy: 0.9195 - loss: 0.2300 - val_accuracy: 0.9301 - val_loss: 0.2124
Epoch 38/40
78/78 ————— 46s 545ms/step - accuracy: 0.9178 - loss: 0.2215 - val_accuracy: 0.8658 - val_loss: 0.4333
Epoch 39/40
78/78 ————— 48s 569ms/step - accuracy: 0.9157 - loss: 0.2322 - val_accuracy: 0.9093 - val_loss: 0.2447
Epoch 40/40
78/78 ————— 51s 612ms/step - accuracy: 0.9209 - loss: 0.2039 - val_accuracy: 0.9187 - val_loss: 0.2556
```

The lowest training loss occurred in Epoch 40, with a value of 0.2039. And, the lowest validation loss occurred in Epoch 34, with a value of 0.2122. **So, Epoch 34 would be the optimal epoch in terms of validation performance.**

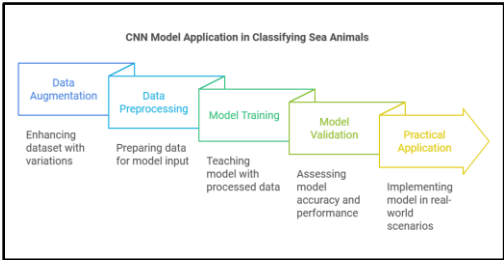
The model achieved **91.20% accuracy on the training dataset**. This indicates that the model is performing quite well in learning from the training data. The **training loss is 0.2407**, which is relatively low. A lower loss value means the model's predictions are close to the actual values in the training set, and the model is learning effectively. The **model achieved 93.57% accuracy on the validation set**, which is **higher than the training accuracy**. This is an excellent sign, as it suggests that the model is not only learning well on the training data but is also generalizing effectively to unseen data. **The validation loss of 0.2122 is even lower than the training loss**, which is a positive indication. A lower validation loss means the model is making fewer errors on the validation data, which further supports the model's ability to generalize well to unseen examples.



Classification Report:				
	precision	recall	f1-score	support
Dolphin	0.9223	0.8051	0.8597	118
Jelly Fish	0.9138	0.8346	0.8724	127
Turtle_Tortoise	0.8878	0.9685	0.9264	286
accuracy			0.9002	531

The model is performing well overall, with an accuracy of 90.02%.

The augmented model outperforms the base model in terms of validation accuracy and loss, indicating better learning and generalization to unseen data. Data augmentation significantly contributed to this improvement, addressing key issues of class imbalance and overfitting.



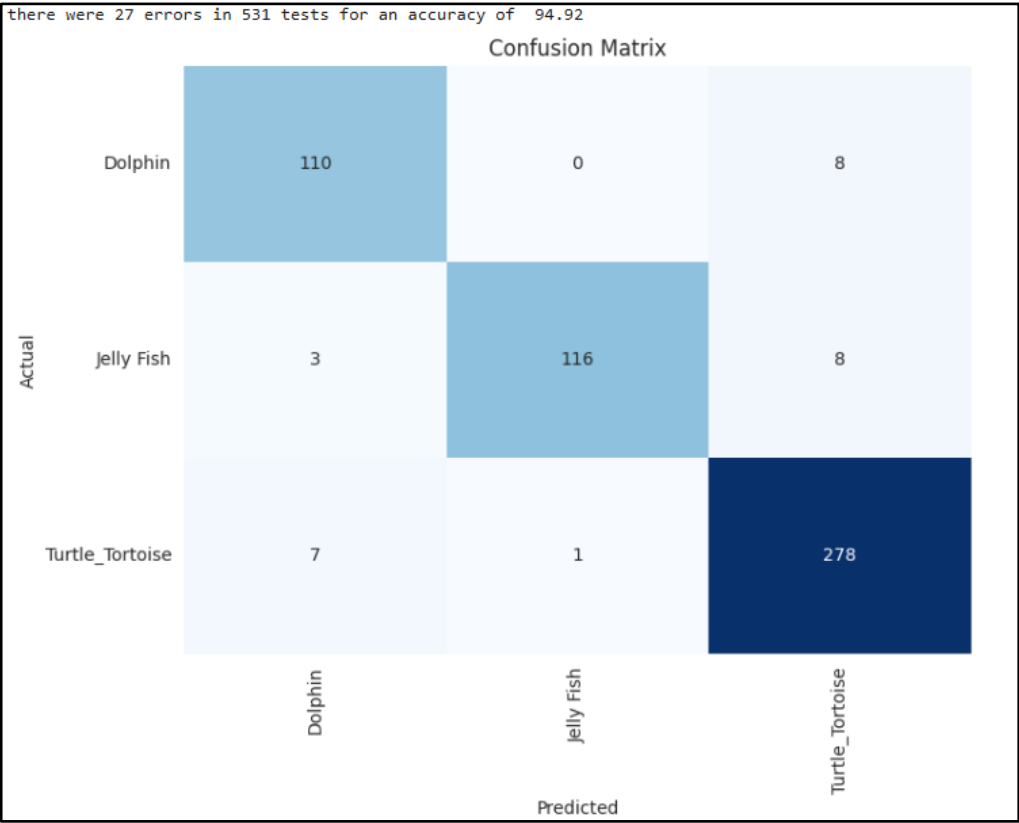
Transfer Learning

Transfer Learning is employed by incorporating the **EfficientNetB7** architecture, pre-trained on the ImageNet dataset, as the base of the model. EfficientNetB7 acts as a feature extractor, leveraging its pre-learned representations from a large-scale, diverse dataset to provide a strong foundation for the current classification task. In this experiment, the model training is enhanced with two callback functions: **EarlyStopping** and **ModelCheckpoint**.

The EarlyStopping callback monitors the validation loss (val_loss) and stops training if there is no improvement in the validation loss for 5 consecutive epochs. This helps prevent overfitting and unnecessary training time, as training halts when the model has plateaued. Additionally, the restore_best_weights=True option ensures that the model's weights are reverted to the best-performing state based on the lowest validation loss.

The ModelCheckpoint callback is used to save the model's weights at the epoch with the lowest validation loss. This ensures that the best-performing model (in terms of validation performance)

is saved for future use or deployment, regardless of the model's performance in subsequent epochs.



Classification Report:				
	precision	recall	f1-score	support
Dolphin	0.9167	0.9322	0.9244	118
Jelly Fish	0.9915	0.9134	0.9508	127
Turtle_Tortoise	0.9456	0.9720	0.9586	286
accuracy			0.9492	531

The model performs very well overall, with high precision, recall, and F1 scores for all classes. It performs exceptionally well in detecting the Turtle/Tortoise class, with the highest recall (97.20%) and F1-score (95.86%). While the model performs well for Jellyfish, the recall is slightly lower than for Turtle/Tortoise, but precision remains excellent. The Dolphin class, while lower in recall compared to Turtle/Tortoise, still performs well with a good F1 score.

The overall accuracy of 94.92% demonstrates the model's strong performance in multi-class classification.

```
Validation Loss over epochs:
Epoch 1: Validation Loss = 10.968378067016602
Epoch 2: Validation Loss = 4.64564323425293
Epoch 3: Validation Loss = 3.124537467956543
Epoch 4: Validation Loss = 1.9531962871551514
Epoch 5: Validation Loss = 62145.94921875
Epoch 6: Validation Loss = 4.026369094848633
Epoch 7: Validation Loss = 2.0255162715911865
Epoch 8: Validation Loss = 4.061318397521973
Epoch 9: Validation Loss = 0.7472755312919617
Epoch 10: Validation Loss = 0.49998149275779724
Epoch 11: Validation Loss = 0.35554733872413635
Epoch 12: Validation Loss = 0.2788771390914917
Epoch 13: Validation Loss = 0.2408534288406372
Epoch 14: Validation Loss = 0.24157917499542236
Epoch 15: Validation Loss = 0.18356581032276154
Epoch 16: Validation Loss = 0.24434694647789001
Epoch 17: Validation Loss = 0.6041899919509888
Epoch 18: Validation Loss = 2.6696724891662598
Epoch 19: Validation Loss = 0.9010420441627502
Epoch 20: Validation Loss = 0.5333196520805359
```

During the training process, the model's validation loss reached its lowest stable value at Epoch 15, with a **validation loss of 0.1835**. This point marks an optimal balance between model learning and generalization, as subsequent epochs showed increased instability in validation loss, with occasional spikes.

Comparison

The comparison between the base model, augmented data model, and transfer learning model reveals notable differences in performance. **The base model** achieved an impressive **99.45% training accuracy and 92.06% validation accuracy**, demonstrating the gap between the training and validation performance, suggesting **limited generalization to the unseen data**.

The augmented data model, on the other hand, improved **validation accuracy to 93.57%**, **effectively eliminated overfitting**, and showed better generalization, as indicated by a lower validation loss of 0.2122. The augmentation techniques, including horizontal flipping, rotation, and zoom, helped mitigate issues of class imbalance.

Finally, **the transfer learning model**, utilizing **EfficientNetB7**, showed **the most significant improvements, achieving an overall accuracy of 94.92%**, with high precision, recall, and F1 scores across all classes, particularly excelling in classifying "Turtle/Tortoise." **This model demonstrated the advantages of leveraging pre-trained features, resulting in superior performance, better generalization, and faster training.**

Technique	Test Accuracy	Validation Loss
Initial Model	88.70%	0.3072
Data Augmentation	90.02%	0.2122
Transfer Learning	94.92%	0.183

Model Application

The model, combined with data augmentation and transfer learning, is highly effective for real-world image classification tasks, particularly when dealing with limited, imbalanced, or low-quality data. By utilizing pre-trained base models like ResNet50 or EfficientNet, the model can quickly adapt to new datasets, drastically reducing training time while enhancing accuracy. Data augmentation further boosts the model's generalization capabilities by creating a more diverse set of training images, which helps prevent overfitting. This powerful combination makes the model especially valuable in fields such as marine biology, conservation, and environmental monitoring.

Additionally, we believe that the developed model could serve as a powerful resource for **scientific research**, aiding in the logging and tracking of various species in **aquatic studies**. It can be utilized to analyze and monitor the population of diverse species, **supporting conservation strategies** through large-scale datasets focused on endangered sea animals. As a valuable asset in the research industry, this model offers conservationists real-time assessments of ecosystem health. Moreover, it empowers students and enthusiasts to learn about and automate species identification and monitoring, further driving the growth of conservation efforts.

Conclusion

This project successfully leveraged advanced deep learning techniques, including transfer learning and data augmentation, to develop an efficient model for classifying sea animals. By using pre-trained models like EfficientNet, the model demonstrated strong performance in identifying key species: dolphins, jellyfish, and turtles, even with limited domain-specific data. The application of data augmentation methods, such as rotation and flipping, enhanced the model's ability to generalize across various environmental conditions, simulating real-world scenarios like underwater imaging. The model's successful implementation highlights its potential in marine

biology and conservation efforts, where it can assist researchers in automating the identification and tracking of sea species. This project not only showcases the power of modern techniques in solving domain-specific challenges but also sets the foundation for future advancements in marine ecosystem monitoring and wildlife protection.

Key learnings from this project include the effectiveness of transfer learning in reducing the need for extensive labeled data and computational resources, and the importance of data augmentation in addressing variability in environmental conditions. We also gained practical experience in fine-tuning hyperparameters, such as the learning rate and the number of epochs, to optimize model performance. Additionally, handling class imbalance through augmented data and understanding how to monitor training progress using validation loss and metrics like accuracy were key takeaways. Overall, the project provided hands-on exposure to the complexities of working with real-world data, where model robustness and generalization are critical for practical applications. The use of deep learning in marine conservation also emphasized the growing role in preserving biodiversity and supporting sustainable environmental research.

Challenges

- The use of cloud-based environments like Google Colab posed challenges with limited GPU availability, which occasionally interrupted the training process. To mitigate this, model training was broken into smaller stages to ensure efficient use of available resources. Additionally, the use of batch processing and reducing the image input size helped manage computational load.
- Overfitting was a concern throughout the project. To address this, techniques such as data augmentation, and early stopping were employed. These strategies significantly improved model generalization, allowing the model to perform well on unseen data.
- The selection of a pre-trained base model, such as ResNet50, helped expedite the model development process. However, fine-tuning the model to meet specific needs, such as adjusting learning rates and layers, was necessary to optimize performance. Hyperparameter tuning required multiple iterations and experimentation to achieve the best validation loss and accuracy.
- Implementing transfer learning helped overcome the challenge of limited training data. Fine-tuning the pre-trained models allowed the model to leverage previously learned features, which were crucial in improving performance without requiring a massive dataset.
- A significant challenge was managing the time spent on model training and testing. The iterative nature of deep learning meant that results were sometimes unpredictable, so allocating time for model experimentation and evaluation was critical for ensuring the quality of the final model.

- Despite the challenges faced during the project, the final model demonstrated strong performance with minimal validation loss and high accuracy. The model's ability to classify sea animals accurately has practical applications in marine biology and conservation efforts, providing a tool for automated monitoring and research in these areas.

References

Vencerlanz, A. (n.d.). Sea Animals Image Dataset. Retrieved December 11, 2024, from <https://www.kaggle.com/datasets/vencerlanz09/sea-animals-image-dataste?select=Clams>

TensorFlow. (n.d.). An End-to-End Open Source Machine Learning Platform. Retrieved December 11, 2024, from <https://www.tensorflow.org/>

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. Retrieved December 11, 2024, from <https://arxiv.org/pdf/1712.04621>

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. Retrieved December 11, 2024, from https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on Image Data Augmentation for Deep Learning. Retrieved December 11, 2024, from <https://arxiv.org/pdf/1905.11612.pdf>

Pan, S. J., & Yang, Q. (2010). A Survey on Transfer Learning. Retrieved December 11, 2024, from <https://ieeexplore.ieee.org/document/5288526>

National Oceanic and Atmospheric Administration (NOAA). (n.d.). Marine Life. Retrieved December 11, 2024, from <https://www.noaa.gov/education/resource-collections/marine-life>